

BACK-END DEVELOPER CHO WONHEE

백엔드 개발자 조원희입니다.
더 나은 사용자 경험을 만들기 위해, 오늘도 한 줄의 코드를 고민합니다.
작은 개선이 쌓여 큰 변화를 만든다고 믿습니다.



하루하루

하루를 공유하는 3일 소개팅 (소프트웨어 마에스트로)

6명의 참가자가 정해진 시간마다 영상을 공유하고,
최종 상호 선택 시 1:1 채팅으로 연결됩니다.

Challenge :

A : 운영 경험을 제품 기능으로 전환

- 시간당 업로드 현황을 확인하는 관리자 대시보드 구현
- 미션 예약 배정과 방별 수동 전송 기능 제공
- 참여가 모였던 시간대를 타임 슬롯 업로드 규칙으로 반영

B : 결제와 코인 데이터 정합성

- Webhook의 eventId와 transactionId를 기준으로 중복 지급 차단
- 지갑 행 잠금과 코인 원장 기록을 함께 처리해 Lost Update 방지
- 구매와 환불 결과를 원장에 기록해 변경 내역을 추적 가능하게 관리

PROJECT INFO

Year : 2026.06 ~ 2026.09

Role : Back-End

Skill : Java, Spring, PostgreSQL, AWS, Docker

PROJECT #1



PROJECT #1

앱을 만들기 전, 120명의 반응을 먼저 확인했다

유행중인 다른 어플을 통해 3대3 소개팅을 직접 운영하며 타겟과 운영 방식을 검증했다.

01 / 처음 가설

페르소나를 20대 중후반으로 시작했지만
다시 오는 사람은 적었다

현재 유행중인 셋로그 어플에서
3대3, 3일 소개팅을 운영했다.
다만 다음 기수에 다시 참가한 사람은 12명 중 1명이었다.

재참여율 8.3%

02 / 타겟 전환

영상 공유 방식에 맞춰
대학생으로 타겟을 바꿨다

영상 콘텐츠가 소개팅보다 자만추에 가깝다고
판단했다. 이후 전남대 에브리타임을
중심으로 참가자를 모집했다.

재참여율 30%

기존 8.3%에서 약 3.6배 상승했다.
20시 동시 업로드 미션은 약 90%가 참여했다.

03 / 운영 결과

10기까지 운영하며
앱을 만들 근거를 얻었다

20개 방 / 실제 참여자 112명
무료 3기 → 보증금 3기 → 유료 4기
보증금과 유료 운영 이후 이탈자 0명

최종 선택률 83.7%

상호 매칭률 37.2% / 하루 평균 업로드 6.8건
참가비 6,900원 / 누적 수익 약 18만원

운영하며 알게 된 것을 기능으로 옮겼습니다

시간당 업로드 확인
→ 관리자 대시보드

방별 미션 운영
→ 미션 자동 배정과 수동 전송

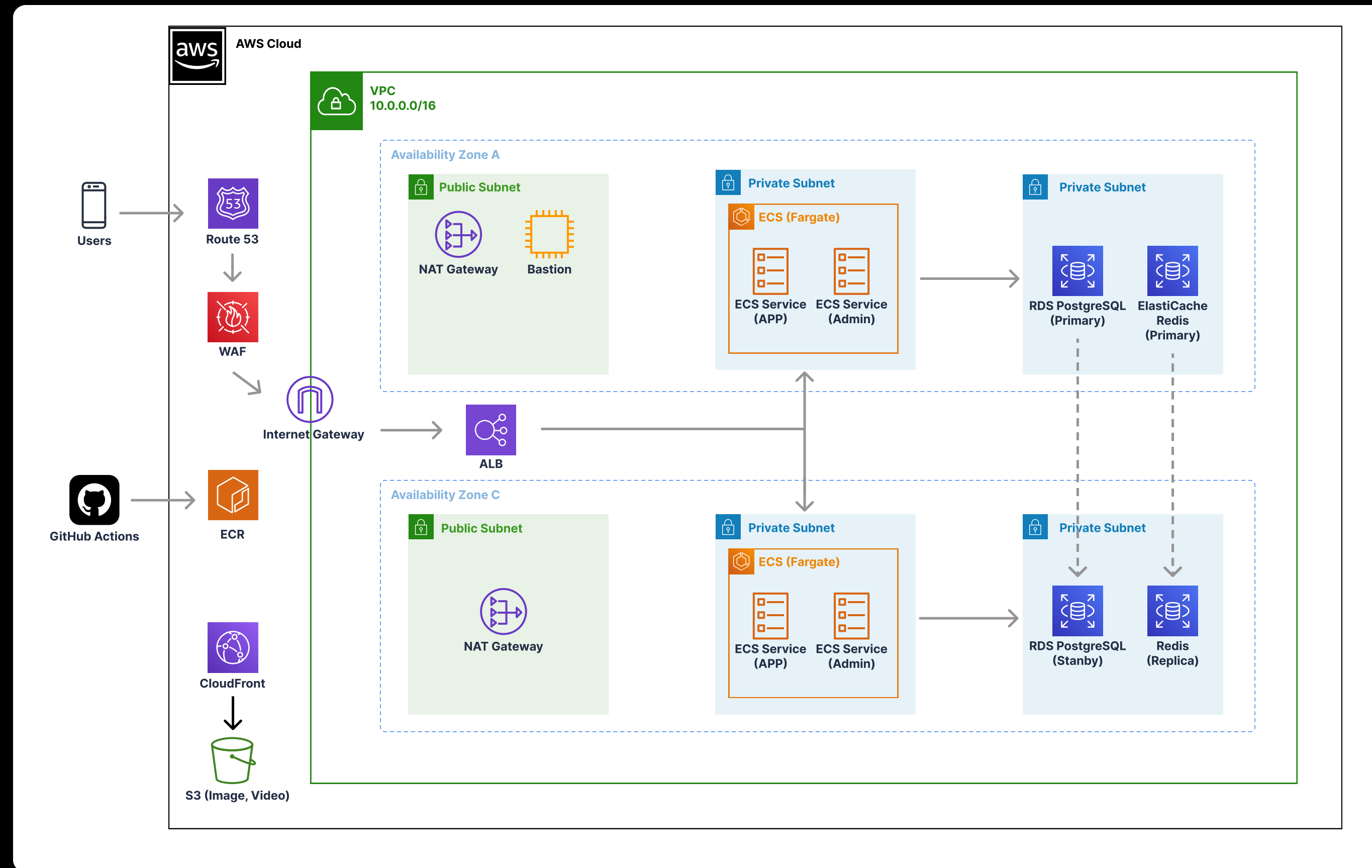
정해진 시간에 함께 업로드
→ 타임 슬롯 업로드

배운 점 : 운영을 먼저 해보니, 어떤 기능이 필요한지보다 왜 필요한지가 더 명확해졌습니다.

PROJECT #1

서비스 가용성과 보안을 고려한 AWS 인프라 설계

진입점, 컴퓨팅, 데이터, 미디어의 역할과 장애 범위를 분리하고, 각 계층을 독립적으로 운영할 수 있도록 구성했다.



Network : Route 53 → WAF → ALB로 외부 진입점을 단일화하고, ECS, RDS, Redis는 Private Subnet에 배치해 직접 노출을 차단했다.

Availability : ECS와 데이터 계층을 두 AZ에 분산하고 PostgreSQL Primary/Standby, Redis Primary/Replica로 단일 장애 지점을 줄였다.

Operation : Fargate와 GitHub Actions OIDC → ECR → ECS로 배포를 단순화하고, S3 직접 업로드로 미디어 부하를 API 서버에서 분리했다.

PROJECT #1

RevenueCat 중복 Webhook과 지갑 동시성 문제를 데이터 무결성으로 해결

결제 이벤트는 재전송될 수 있고, 지갑 잔액은 여러 요청에 의해 동시에 변경될 수 있다.
중복 지급과 Lost Update를 방지하기 위해 결제 식별자, DB 제약, 행 잠금을 하나의 처리 흐름으로 설계했다.

문제 상황 (Problem)

1) 반복 전송되는 결제 Webhook

RevenueCat은 응답 실패나 네트워크 상황에 따라서 동일 구매 이벤트를 다시 전송할 수 있다.
처리를 반복하면 같은 구매에 코인이 여러 번 지급될 위험이 있다.

2) 지갑 잔액의 Lost Update

동시 요청이 같은 잔액을 읽고 각각 저장하면
한쪽 변경 결과가 사라질 수 있다.

Ex) INITIAL 100
+10 / -5 → 95 or 110
EXPECTED → 105

해결 과정 (Solution)

1) Webhook 이중 멱등성

- eventId UNIQUE로 동일 이벤트 재시도 차단
- storeTransactionId UNIQUE로 동일 거래 차단
- 애플리케이션과 DB에서 이중 방어

2) 사용자별 지갑 행 잠금

- SELECT FOR UPDATE로 지갑 한 행만 잠금
- Read-Modify-Write 구간을 직렬화
- 잔액 변경과 Ledger 기록을 함께 커밋

3) 이벤트별 처리 결과 연결

- PURCHASE → 적립 · CANCELLATION → 회수
- 미지원 이벤트 → 기록만 유지

APP PRE-CHECK + DB UNIQUE + ROW LOCK

검증 결과 (Result)

1) Webhook 통합 테스트

동일 이벤트를 2회 전송해도
Wallet 120, Purchase 1, Event 1
잘못된 Secret은 401, 데이터 변경 0

2) 20개 스레드 동시 적립

20 Threads × +10
최종 잔액 200, Ledger 20, 실패 0
Lost Update 0

3) 적립·차감 혼합 검증

초기 1,000 + 10×(+10) + 10×(-5)
최종 잔액 1,050
Wallet Balance = Σ Ledger

메타버스 플랫폼 프로젝트 ON

비대면 학습의 낮은 몰입도와 소속감 부재를 해결하기 위한 메타버스 기반 플랫폼
가상 공간에서의 실시간 아바타 상호작용을 통해 오프라인 수준의 동질감과 학습 동기를 부여합니다.

Technical Challenge :

A : RDB의 구조적 한계와 실시간성의 충돌

- 빈번한 Write 트랜잭션으로 Disk I/O 및 Row Lock 경합 발생
- 동시 접속 100명 좌표 동기화 시 최대 5.6초의 지연 발생
- 기존 아키텍처 한계를 극복하며 0.2초 미만의 저지연 동기화

B : 비정상 종료와 동시성(Concurrency) 제어

- 브라우저 강제 종료(비정상 이탈) 시 세션 잔존 현상 해결
- 0.01초 차이의 동시 입장 요청 시 정원 초과 문제 방지
- Stateless 환경에서 세션의 원자성(Atomicity) 및 데이터 정합성 보장

PROJECT INFO

Year : 2025.08 ~

Role : Back-End

Skill : Java, Spring, Redis, PostgreSQL

ON

login



Learn Together, Live Together

ON

단순한 채팅이 아니라, 같은 공간에 함께 있다는 존재감으로 학습 동기부여를 받아보세요.

시작하기

ON

Welcome nickname!

logout

Join Room !

입장 가능한 방을 선택해 주세요

Q 검색하기

code

방이름
총 1/4방이름
총 1/4방이름
총 1/4방이름
총 1/4방이름
총 1/4방이름
총 1/4

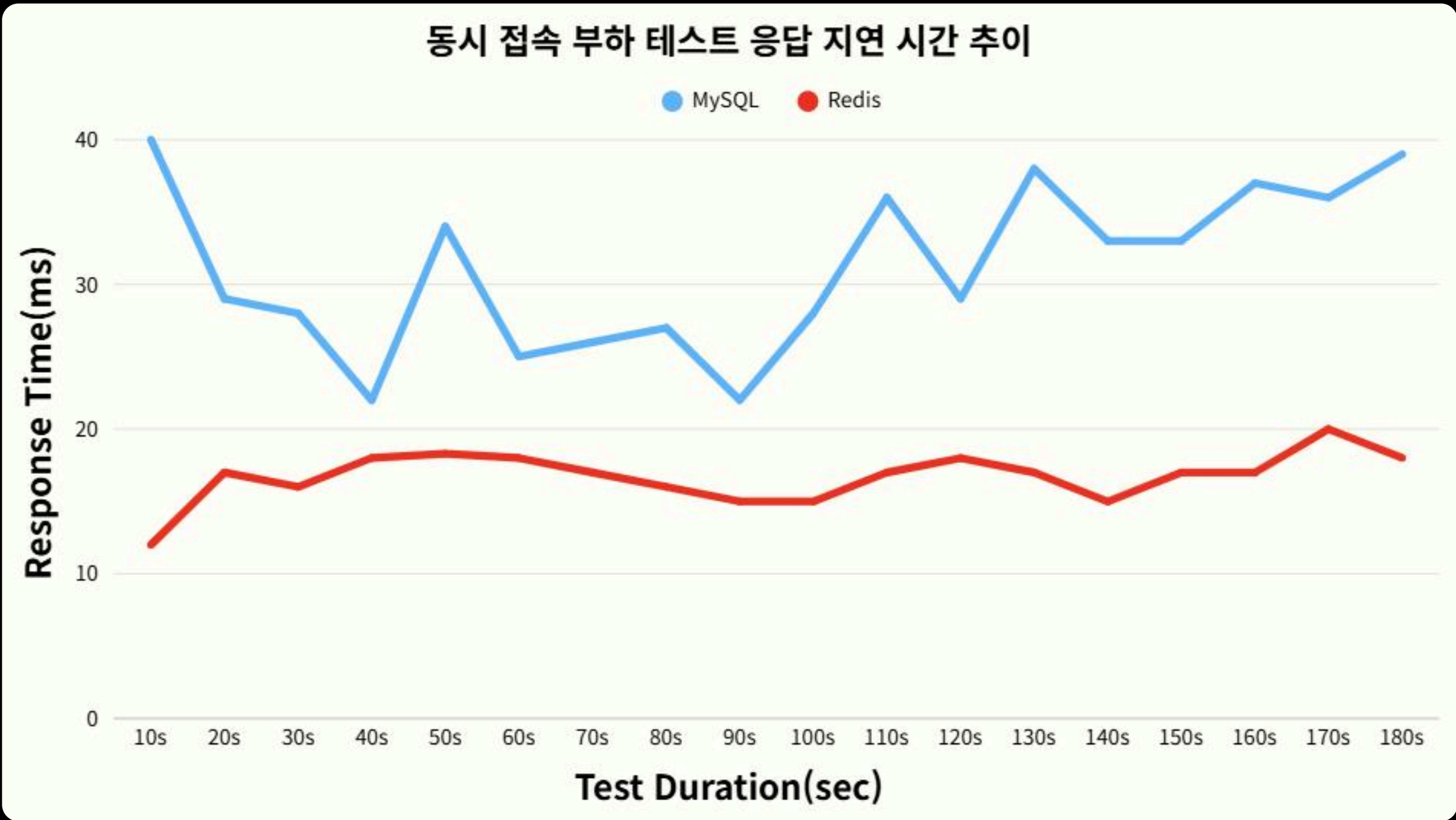
•••••



PROJECT #2

RDB의 Disk I/O 병목으로 인한 5.6초의 지연, In-Memory 기반 Redis로 97% 개선

동시 접속자 100명이 0.1초마다 움직이는 혼잡 상황을 가정하여 부하 테스트를 진행할 때, MySQL의 경우, 잦은 Update로 인해 평균 응답 속도는 30ms가 나왔고, 최대 지연 시간(Max Latency)은 5,646ms(5.6초)가 발생했다.



비교 항목	MySQL	Redis	Impact
평균 응답 속도	30ms	17ms	약 2배 향상
최대 지연 시간	5,646ms	126ms	약 45배 개선
초당 처리량	671/sec	769/sec	약 15% 증가
표준 편차	36.20	9.85	3.6배 안정적

디스크 I/O가 발생하는 RDB 대신, 필드 단위의 O(1) 접근 속도로 잦은 상태 변경에 최적화된 Redis Hash 구조를 도입할 경우 평균 응답 속도는 17ms로 약 2배 가량 단축되며, 중요한 최대 지연은 126ms로 약 45배 개선된다.

이러한 Redis의 성능 처리 덕분에, 5초마다 방 내 모든 유저의 위치를 조회하여 전송하는 '스냅샷 동기화 (Snapshot Sync)' 로직을 지연 없이 구현할 수 있었다. 이를 통해 네트워크 딜레이로 발생할 수 있는 클라이언트 간의 위치 오차를 주기적으로 보정하여, 데이터 적합성을 확보할 수 있었다.

PROJECT #2

Redis 데이터 정합성과 동시성 문제를 원자적으로 해결하다

고아 데이터 발생 원인을 추적하고, Lua Script로 방 생성, 입장 과정을 하나의 연산으로 묶었다.

전체 문제사항 (Problem)

만료된 방이 목록에 남고, 접속 시 오류가 발생했다.

방 메타데이터와 멤버 상태 데이터의 만료 시점이 달라 Redis 데이터 정합성이 깨진 것이 원인이었다.

문제의 이유

rooms:{id} 만료와
멤버 좌표, 상태 키 잔존

문제사항 (Problem) 방 생성과 입장 과정이 여러 Redis 명령으로 나뉘어 있어, 중간 실패나 동시 요청 발생 시 일부 데이터만 반영되거나 정원을 초과할 가능성이 있었다.

해결 과정 (Solution)

방 생성과 유저 입장 과정에서 발생할 수 있는 불일치를 Redis 내부의 원자적 연산으로 차단했다.

01 방 생성

연관 키 생성과 TTL 동기화

메타데이터, 멤버 리스트, 위치 정보를 한 번에 생성
모든 연관 키에 동일 TTL을 적용해 데이터 생명주기 일치

02 유저 방 입장

입장 검증과 상태 변경 원자화

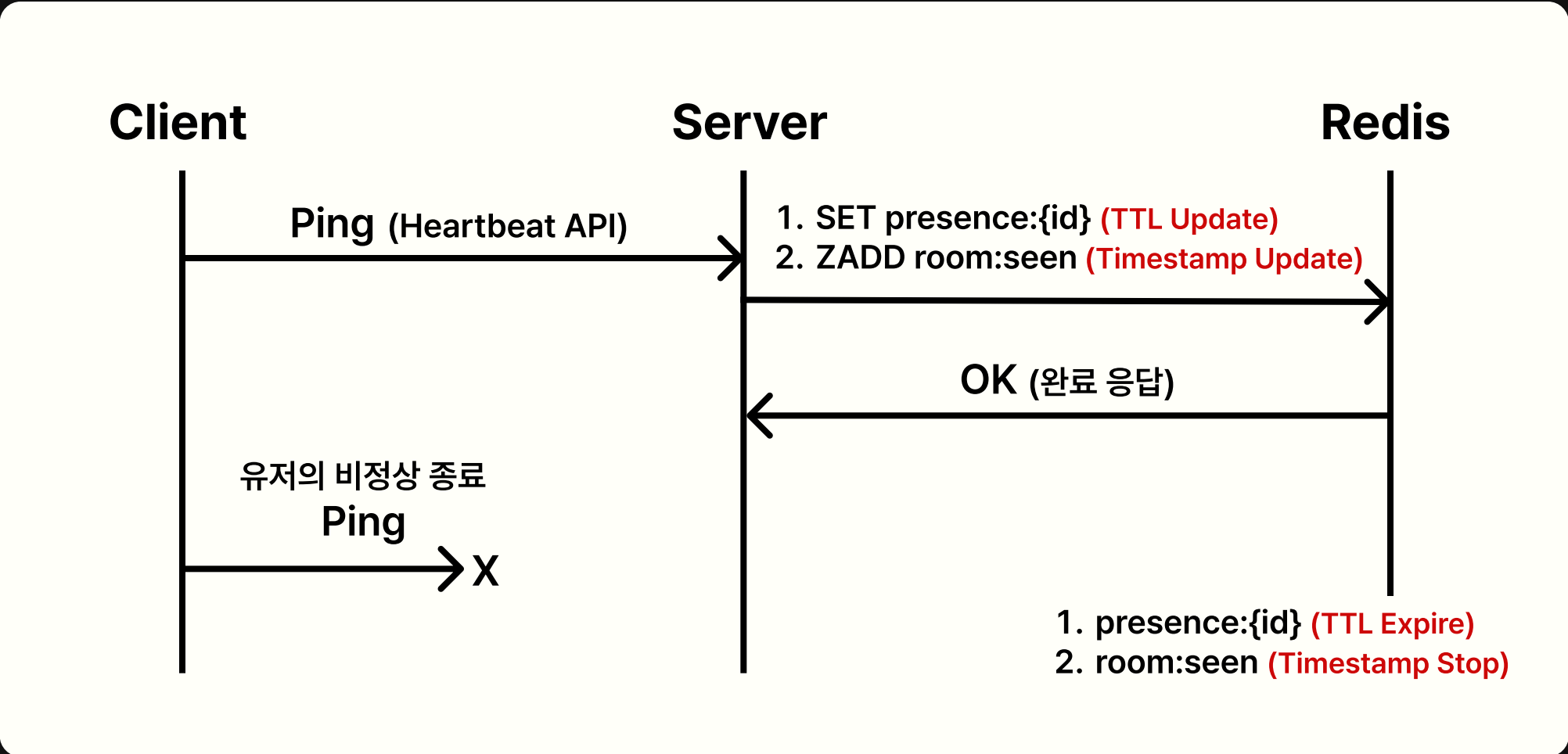
방의 존재, 활성화 상태와 정원이 가득 찼는지를 Redis 내부에서 검증
정원 확인과 인원 증가를 하나의 연산으로 묶어 동시 입장 충돌 방지

PROJECT #2

문제사항 (Problem) 사용자가 비정상 종료하면 퇴장 요청이 전달되지 않아 세션과 사용자 정보가 남았다.

해결 과정 (Solution)

Hearbeat(Ping)을 이용한 유저 세션 유지



유저가 브라우저를 비정상 종료(강제 종료, 탭 닫기)할 경우, 퇴장 요청 불가 이를 해결하기 위해 **Heartbeat** 메커니즘을 도입했다.

- Logic : 클라이언트는 주기적으로 서버에 Ping을 보낸다.
- Action : 서버는 유저 세션 키의 TTL을 갱신한다.
- Result : Ping이 멈추면 해당 키가 만료되어 **"Zombie User"**로 간주된다.

유저 세션 관리 구현 로직

```
// Zombie User Cleanup Scheduler

@Scheduled(fixedDelay = 120000)
public void cleanupExpiredUsers() {
    // TTL 만료 유저 조회 (Redis)
    Set<String> expiredUsers = presenceRepository.findExpiredUserInRoom(roomId);

    for (String userId : expiredUsers) {
        // 데이터 정리 (Redis Del)
        roomCleanUpRepository.cleanupExpiredUser(roomId, userId);

        // 실시간 퇴장 알림 전송 (STOMP Broadcast)
        // 유저가 비정상 종료하더라도, 다른 유저들에게는 '퇴장'으로 처리됨
        template.convertAndSend("/topic/room/" + roomId, new LeaveDTO(userId));
    }
}
```

Redis TTL(Key Expiration)은 데이터는 지워주지만, “해당 유저가 나갔다”는 사실은 실시간으로 방의 다른 사람들에게 알릴 수 없다.

1. **@Scheduled**를 이용한 Batch 작업 수행.
2. Redis에서 만료 대상(expiredUserIds)을 조회
3. 해당 유저 데이터를 삭제함과 동시에 STOMP로 퇴장 메시지를 브로드캐스팅하여 클라이언트 동기화 유지

PROJECT #2

문제사항 (Problem) Redis의 TTL이 만료돼도 방 종료와 사용자 퇴장 상태가 실시간으로 반영되지 않았다.

해결 과정 (Solution)

방 및 세션 만료 처리

전략 : Event-Driven과 Scheduling을 결합한 이중 안전장치

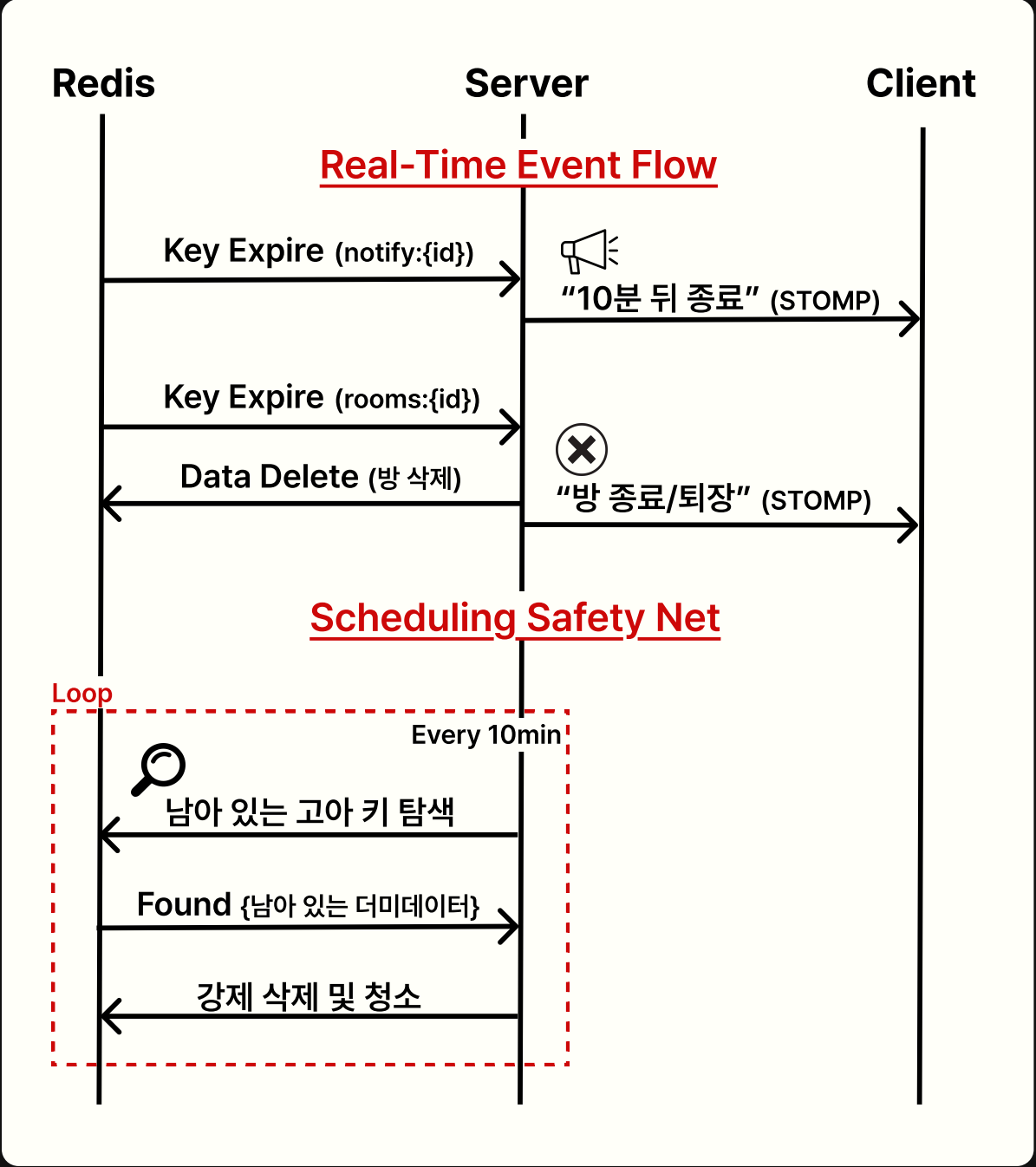
1) 실시간 처리 (Redis Keyspace Notification)

- Role : 즉각적인 상태 변화 감지 및 알림
- Flow : Key 만료 이벤트(__keyevent@00__:expired) 수신 → STOMP 브로드캐스팅
- Action : 종료 10분 전 알림 (notify:{id}), 방 완전 종료 및 삭제 (rooms:{id})

2) 데이터 정합성 보장 (Cleanup Scheduler)

- Role : 이벤트 누락 대비 및 메모리 누수 방지
- Action : 10분 주기 배치 작업을 통해 메타데이터가 없는 고아 키(Orphaned Data) 강제 삭제

Room & Session 만료 처리



3가지의 해결과정을 통한 결과 (Result)

데이터 정합성과 세션 생명주기를 함께 안정화했다.

Lua Script로 방 생성, 입장 로직을 원자화해 동시성 충돌과 고아 데이터 발생 가능성을 줄였다.
또한 Heartbeat와 TTL로 비정상 종료를 감지하고, Keyspace Notification과 Scheduler를 통해 세션을 자동 정리해 실시간 동기화를 유지했다.

KnowWhoHow

PROJECT #3

시니어 대상 노후 대비 자산관리 서비스 (우리FIS Academy 메인 프로젝트)

(Know Who) 고객이 누구인지를 깊이 이해하고
(Know How) 그들에게 딱 맞는 방법을 제시하는 서비스

Challenge :

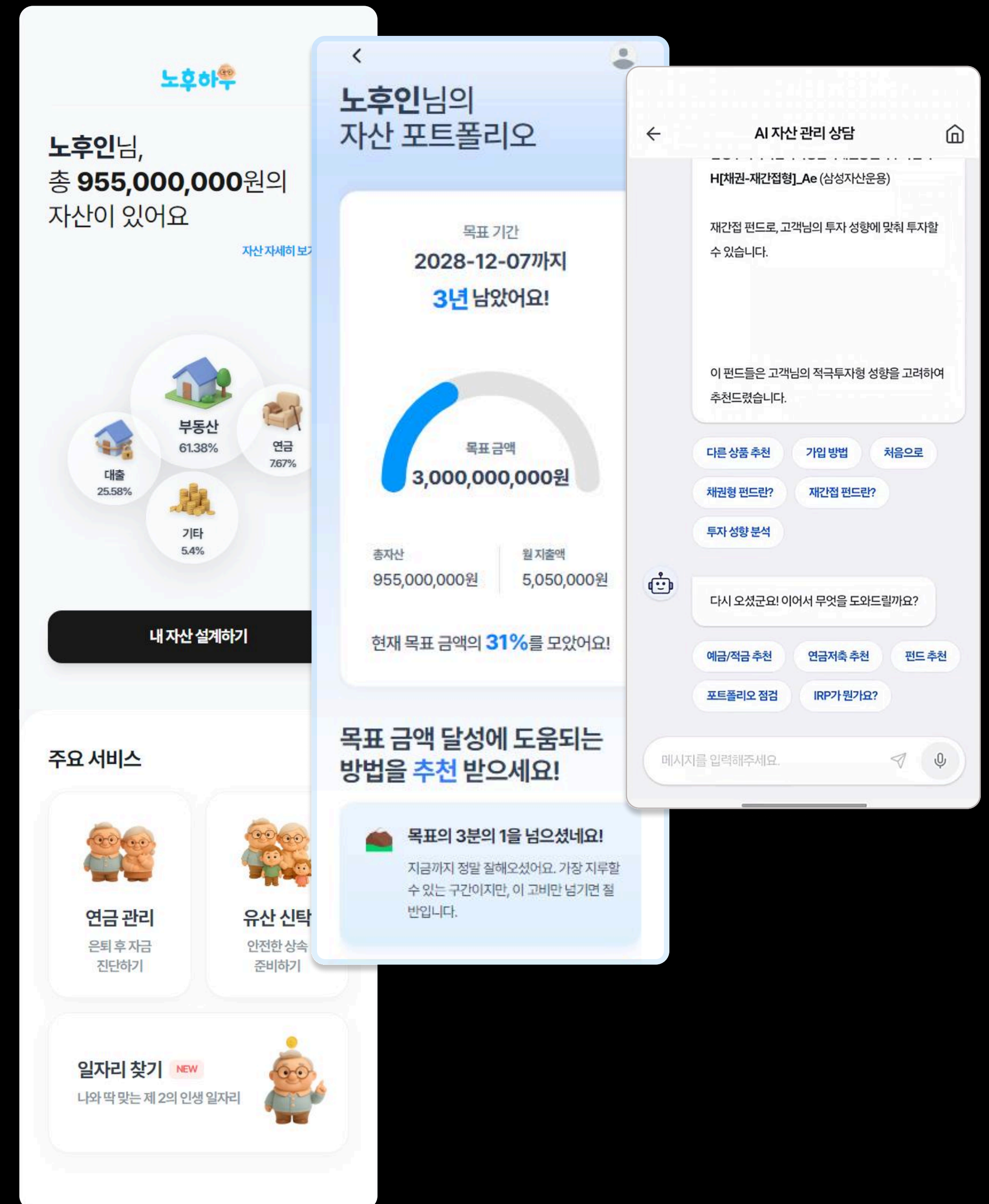
- A. 절대적 시간 제약
 - 7주 내 기획, 디자인, 개발, 배포(CI/CD) 및 최종 시연 완료
- B. 시니어 타겟 특수성
 - 시니어 계층을 위한 직관적인 UX와 AI의 조화
- C. 금융 데이터의 정합성
 - 마이데이터 연동 환경을 모사한 Hybrid Cloud(On-Premise ↔ Cloud) 보안 아키텍처 구현
- D. Agile
 - 애자일한 스프린트 운영으로 데드라인 준수 및 리스크 관리

PROJECT INFO

Year : 2025. 10 ~ 2025. 12

Role : PM & Backend Dev

Skill : Java, Spring, Redis, MySQL,
Jenkins, Docker, On-Premise Server, Cloud Server



PROJECT #3

실제 마이데이터 가이드라인을 기반으로 한 OAuth 2.0 기반의 인증/인가 서버 분리 및 토큰 보안

1) 아키텍처의 배경

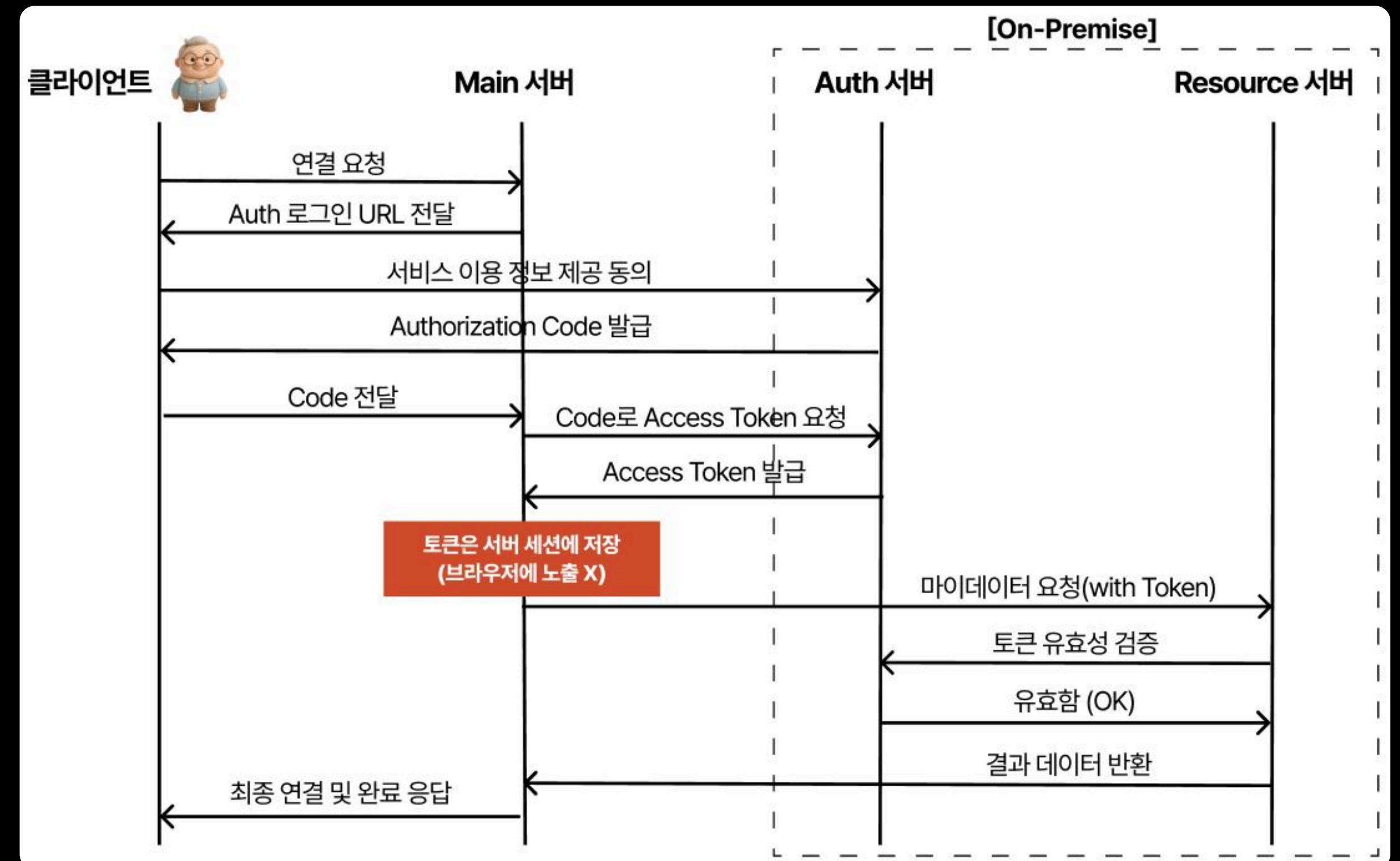
- 실제 금융권 마이데이터 표준을 준수하여, 인증(Auth)와 자원(Resource) 서버를 자체 구현하였다.
- 인증 로직의 독립적 관리와 확장성을 위해 Spring Authorization Server를 별도로 분리

2) 보안 로직 : Authorization Code Grant

- 보안 레벨이 높은 Authorization Code Grant 방식을 채택해, 인증 코드와 Access Token 교환 과정을 백엔드에서 처리
- Resource Server는 Auth Server에서 발급한 유효한 Access Token을 가진 요청에 대해서만 금융 데이터(JSON)를 반환한다.

3) 키 포인트

- Access Token을 브라우저에 노출시키지 않고 Main 서버 세션에 암호화하여 저장
- 클라이언트가 토큰에 직접 접근할 수 없도록 차단하여 XSS 공격 봉쇄

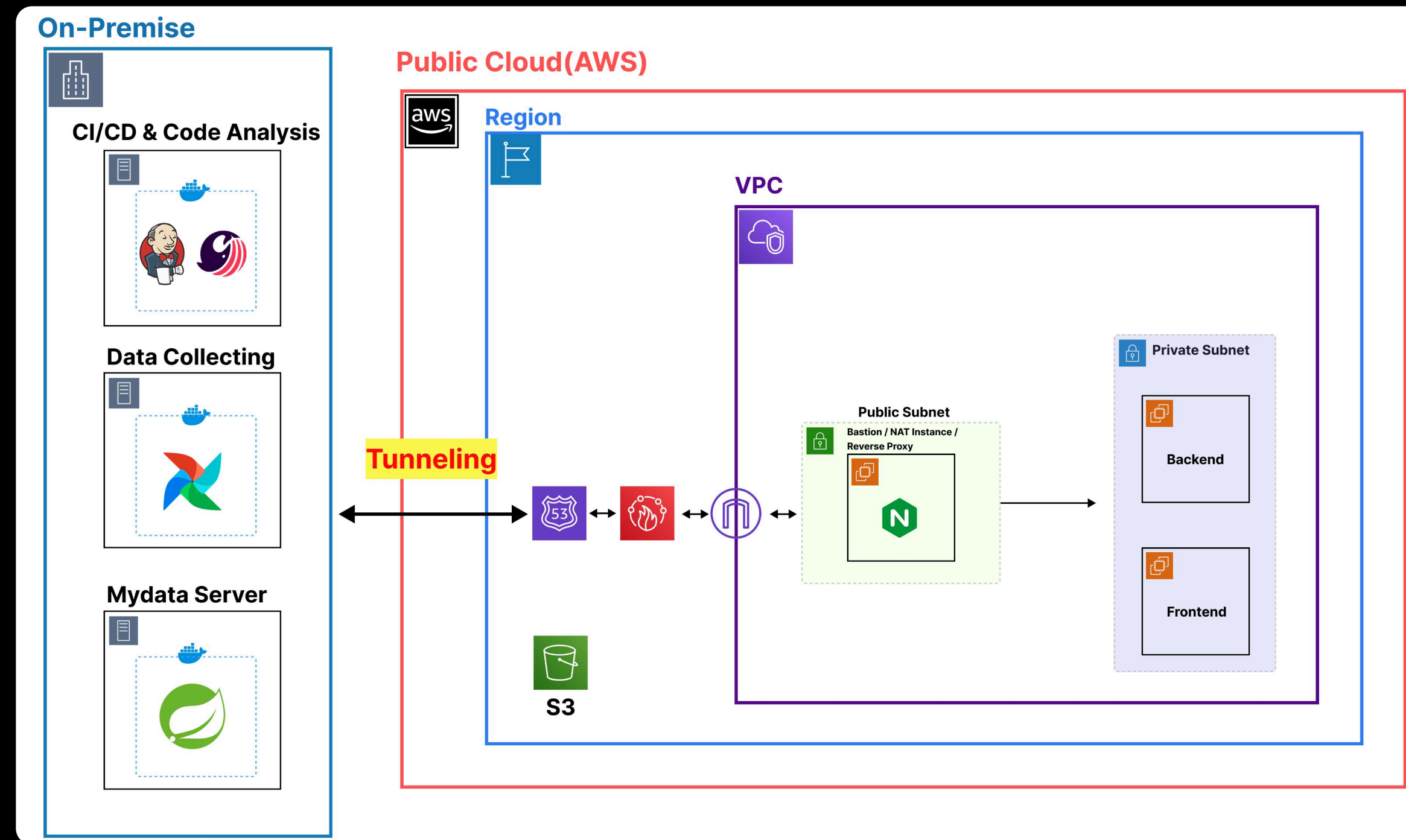


@ Reference : 금융분야 마이데이터 표준 API 규격 (금융위원회·금융보안원)

< OAuth 2.0 인증 흐름 및 토큰 관리 시퀀스 >

PROJECT #3

금융 보안 규정을 준수한 Hybrid Cloud Architecture & DevOps



@ Reference : 전자금융감독규정 제 15조(해킹 등 방지 대책)

- 전자금융감독규정 가이드라인을 참고하여 마이데이터와 같은 민감한 정보는 **On-Premise**에 격리하고 서비스의 접점은 **Cloud**로 구성하여 보안성을 강화했다.

1) Security : 3-Layer Network Isolation

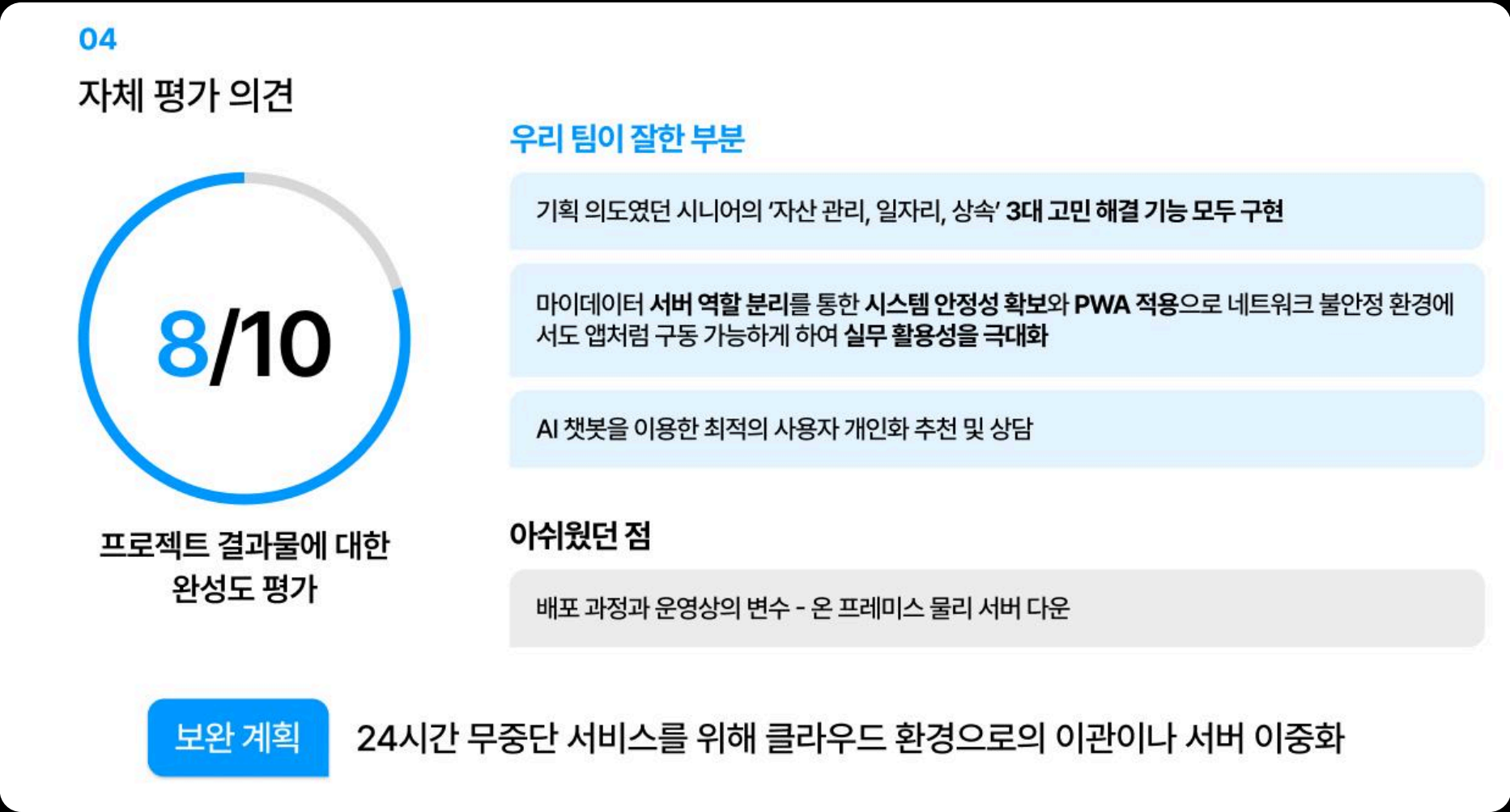
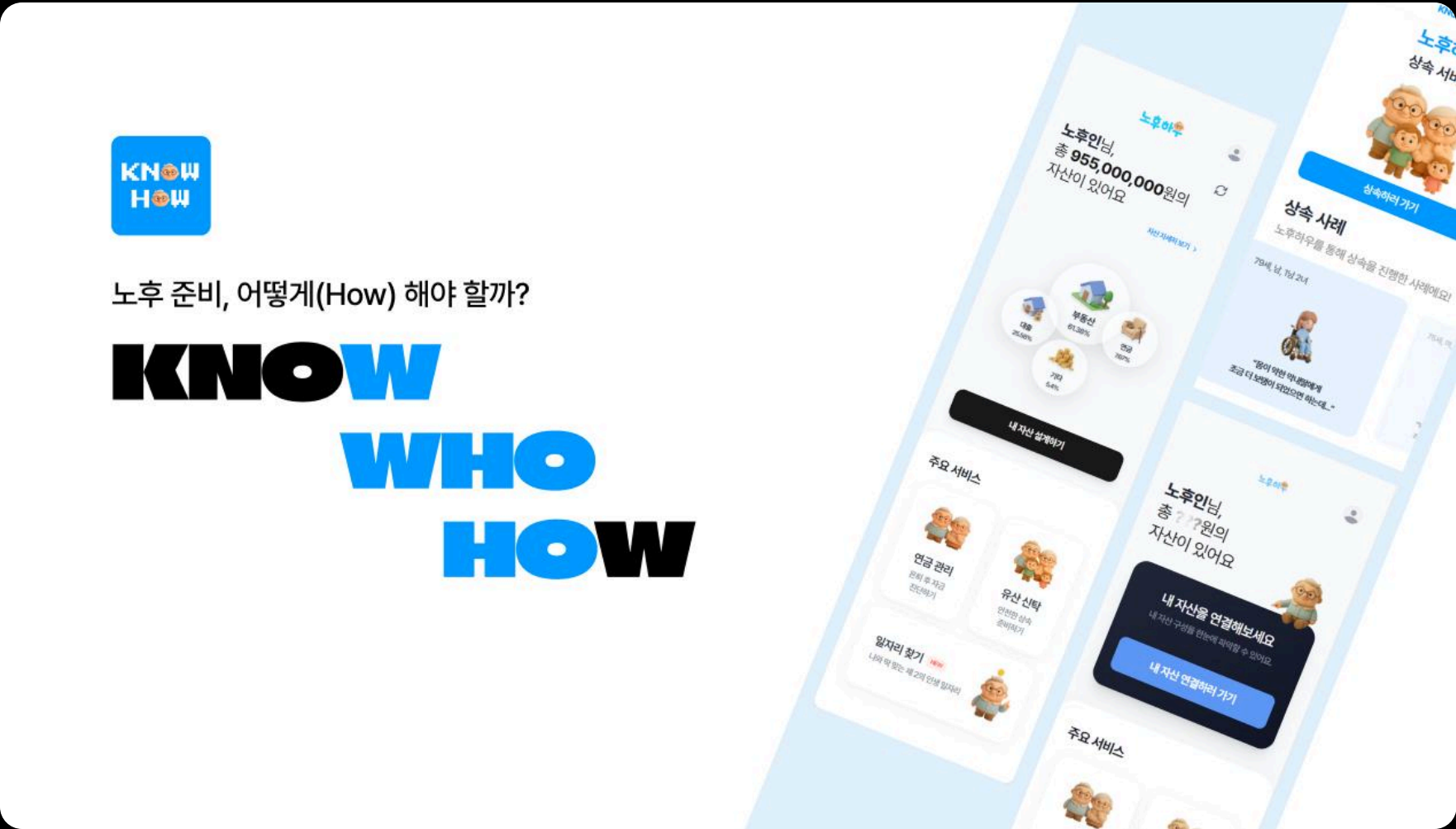
- VPC Segmenataion: Public Subnet과 Private Subnet을 분리하여 외부의 직접 접근을 차단했다 .
- Hybrid Tunneling : On-Premise(내부망)과 AWS(외부망) 구간을 Tunneling으로 연결하여 데이터 전송 보안을 강화했다.

2) DevOps

- CI/CD : 내부망의 **Jenkins**와 **Docker**를 활용하여 클라우드로 배포하는 파이프라인을 구축했다.

PROJECT #3

Project Outcome : 7주간의 몰입이 만들어낸 기술적 성장과 성공적인 완주



*실제 발표 자료 중 일부

1) Final Outcome

- 기획했던 시니어 맞춤형 핵심 기능(자산 포트폴리오, 연금 관리, 상속 설계, 일자리 찾기) MVP 100% 구현 및 배포 완료
- 금융 데이터의 환각 현상 방지를 위한 RAG 기술 도입 및 AI 자산 관리 상담 챗봇 구현

2) Technical Insight

- 팀장의 역할로 마이데이터(OAuth2.0), Redis 이원화 캐싱, AWS S3, RAG 파이프라인 등 프로젝트 핵심 기술 구조 도입을 주도
- 이상적인 기술보다는 프로젝트 마감과 인프라 환경을 고려한 현실적 의사결정의 중요성을 체감하고 팀에 가이드 제시

3) Leadership & Collaboration

- 팀원 이탈로 발생한 공백을 빠르게 파악해 업무를 재분배하고, 핵심 기능을 직접 보완해 일정 지연을 최소화했습니다.

PROFILE



About Me

미국과 일본에서의 어학연수 경험은 단순히 언어를 배우는 것을 넘어,
낯선 환경에서 빠르게 적응하고 다양한 배경을 가진 사람들과 소통하는 법을 가르쳐주었습니다.
이러한 적응력은 새로운 기술 스택을 습득하고,
프로젝트 팀원 간의 의견 차이를 조율하는 팀장 및 팀원의 역할을 수행하는 데 큰 자산이 되었습니다.

EDUCATION

순천향대학교 컴퓨터공학과 2019.03 ~ 2025.08
우리 FIS Academy 2025.07 ~ 2025.12
소프트웨어 마에스트로 2026.04 ~ 2026.09

GLOBAL EXPERIENCE

미국 어학연수 - Kaplan NewYork 2022.11 ~ 2023.05
일본 어학연수 - ISI Language School 2023.06~2024.12

CONTACT ME

Name : 조원희
Phone : 010-2265-5927
Email : woni0517@naver.com
GitHub : <https://github.com/woni-jo>

SKILLS

- Backend : Java, Spring, JPA
- Database & Cache : MySQL, PostGre, Redis
- Infra & DevOps : AWS, Docker, Jenkins, Github Action

CERTIFICATION

- 정보처리기사
- OPIC - IH
- SQLD
- ADsP
- 리눅스 마스터 2급
- 네트워크 관리사 2급